

Furnish Hope — Codebase Summary

Last regenerated: 2026-07-06

What it is

An internal operations platform for a Central Oregon nonprofit that pairs donated furniture with families transitioning into housing. Covers the full lifecycle: donor intake → pickups → inventory → client referrals → matching → delivery, plus fundraising (campaigns, events, pledges, donations → QuickBooks), volunteer management, email, and an in-app issue tracker.

Built specifically for a **non-technical primary user** (a nonprofit's database admin) — every operational page has a friendly UI, but every table is also editable through a generic admin form for power use.

By the numbers

Metric	Count
TypeScript LOC (api + web)	~56,000
Schema migrations	90
Database tables (tbl_* + lkp_*)	82 + 65 = 147
Foreign keys	232
API route files	40
API endpoints	238
Frontend pages	86
Commits on main	138
Production bundle size	~1.9 MB JS, 506 KB gzipped

Stack

Back end (/api)

- **Node 22.x** (pinned in engines) — TypeScript 5.5
- **Express 4** — single service that serves both the REST API and the built React bundle in production
- **PostgreSQL 16/17** via pg (node-postgres) with a custom withTransaction helper
- **express-session + connect-pg-simple** — session store in the same Postgres DB
- **bcrypt** for password hashing
- **express-rate-limit + helmet** — login throttling + standard security headers
- **AES-256-GCM** (via Node crypto) — encryption at rest for OAuth

tokens + IMAP/SMTP passwords

- **Idempotent startup migrations** — `api/src/auth/migrations.ts` runs every migration in order on every boot; safe to re-run
- **node-cron** — schedules the nightly client-dedup scan at 02:00 server time (disable with `DISABLE_DEDUP_CRON=1`)

Front end (/web)

- **React 18 + TypeScript + Vite 5** (build + dev server)
- **React Router 6** — four "shells": staff (main app), agency caseworker portal, public volunteer signup, and the unauthenticated agency-onboarding pages (`/apply-to-refer`, `/referring-agencies`, `/caseworker-register/:token`)
- **Tailwind CSS** + custom design tokens (terracotta / sage / gold / paper palette + serif display font + sans-serif body)
- **TanStack React Query** — every server interaction is a query/mutation with optimistic invalidation
- **FullCalendar** (6 packages) for the unified ops calendar (pickups, deliveries, visits, events, shifts, vendor services)
- **Recharts** for the 16-chart Reports page
- **html2canvas** — captures the visible DOM as PNG for in-app issue reports
- **qrcode** — generates QR codes for printable manifests (driver directions)

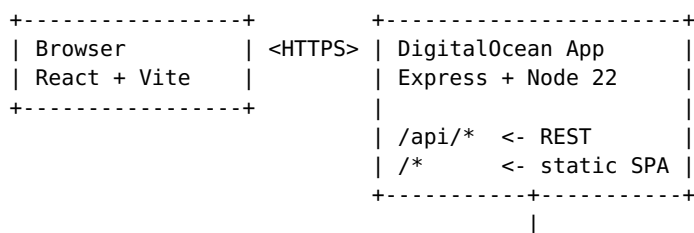
Database (/db)

- `01_schema.sql` — initial schema
- `02_seed.sql` — realistic-looking sample data (donors, clients, etc.) for local dev
- `03_wipe_demo.sql` — `CASCADE TRUNCATE` for production launch day
- ERD generator at `scripts/generate_erd_pdf.py` (Python + Graphviz + `pypdf`) → `docs/FurnishHopeERD.pdf`

Deployment

- **DigitalOcean App Platform** (Node web service) + **Managed Postgres** dev tier
- `.do/app.yaml` describes the deploy as code
- Auto-deploys on every `git push` to main
- `app.set('trust proxy', 1)` in production; sessions become Secure cookies
- Health check at `GET /api/health`
- `~20/monthtotal` (5 App + ~\$15 PG)

Architecture



v

```

+-----+
| DO Managed Postgres |
| furnish_hope DB     |
+-----+

```

Four shells served by the same app:

1. /login, /* — staff app (default)
2. /agency/* — partner-agency caseworker portal (no internal data visible)
3. /volunteer, /volunteer-agreement — public volunteer signup (unauthenticated)
4. /apply-to-refer, /referring-agencies, /caseworker-register/:token — public partner-agency application, marketing list, and caseworker self-serve signup (unauthenticated; token IS the auth on the register page)

Auth + permissions

Role tiers enforced by middleware on every route:

Middleware	What
requireUser	Session must be authenticated
requireStaff	Excludes agency users (role === 'agency')
requireAgency	Only partner-agency caseworkers
requireProgramManager	is_admin = true OR is_program_manager = true (approve/reject agency applications)
requireAdmin	is_admin = true
requireDeveloper	is_admin = true AND is_developer = true

Plus row-level scoping for agency caseworkers (they only see their own org's referrals) and per-user scoping for email accounts and mailbox (you only see your own mail). Public router families (/api/public/*) are mounted BEFORE requireUser, are rate-limited + honeypot-guarded, and expose only the specific endpoints the unauthenticated pages need.

Audit log records every mutation (tbl_audit_log) with field-level diffs. Sensitive fields (passwords, OAuth tokens, encrypted IMAP creds, screenshot blobs, lock codes, DOB, tax IDs) are masked to ***.

Feature domains (12 themes from the ERD)

1. **Contacts & Addresses** — shared people + place rows; address dedupe + shared-edit confirmation modal
2. **Clients & Households** — referral → intake → **packing list** → visit → waiver → delivery. The **packing list** (tbl_client_provisioning_request, UI-labeled "Packing List") is the redesigned pull-and-pack document: a FH-##### reference code, an **Approved Referral** picker that links the list to a specific FH-approved-agency referral (persisted as referral_id),

delivery/pickup logistics for the crew, per-household demographics (tbl_count / adult_female_count / adult_male_count + a row per child in tbl_request_child) that feed the Impact Data report, situation tags/notes, and a room-grouped item checklist (tbl_client_request_items gained item name, room, pulled, qty_given, is_na, is_declined, sort_order) with per-line pull/pack tracking. New lists pre-load from an admin-editable home template (tbl_packing_template_room / tbl_packing_template_item, 13 rooms / ~106 items). Households can be multi-type (Veteran + Disaster + ...) via tbl_client_client_type. Multiple agencies can refer the same client; staff see the full referral history per client. Client-dedup search ("do you mean...?") surfaces likely matches before a new client is created, scoped per-agency in the partner portal. A **nightly dedup scan** (node-cron) scores every pair of clients on name / DOB / phone / email / address and queues high-score pairs to tbl_potential_duplicate; admins resolve via a side-by-side compare + atomic merge that introspects every FK to tbl_client and reassigns rows from the merge side to the keep side in one transaction. Agency-submitted packing lists land in a staff **review queue** with Approve / Edit / Reject actions before joining the matching pipeline. Fulfillment methods: home / walkout / container pickup.

3. **Donors, Donations & QuickBooks** — gifts, designations, pledges, full QBO sync (donations → Sales Receipts), 6 QBO tables for OAuth + mappings + sync log
4. **Inventory & Storage** — items, reservations, container/lockbox flagging
5. **Facilities & Vehicles** — corporate sites, vehicle fleet + maintenance log
6. **Staff, Volunteers & Shifts** — public signup queue, approval flow, profile prefs (availability / activity / physical), shift templates + generated shifts, preference-aware shift signup picker
7. **Fundraising** — campaigns, events with attendees + sponsors, grants
8. **Vendors** — outside service providers + service log
9. **Partner Agencies** — self-serve agency onboarding pipeline: public /apply-to-refer form → PM review queue at /agencies/applications → atomic approval creates the agency + caseworker invitation tokens → caseworker signs up at /caseworker-register/:token → lands in /agency/*. The enhanced caseworker dashboard shows KPIs (this-month referrals + total + open + delivered), request status pills, a merged activity feed (referrals + requests + deliveries), and a team table with Active / Invited-until-DATE / Expired / Revoked pills. Every /api/agency/* endpoint scopes by req.user.agency_id; admin dropdowns use a table-level fkOptionsFilter so unapproved agencies (and their contacts) can't be picked for new referrals even while existing rows continue to display their agency name.
10. **Communications, Files & Notes** — per-user email accounts (IMAP/SMTP), Mailbox view, email templates, generic per-entity attachments
11. **System** — user accounts, audit log, app settings, in-app issue tracker, broadcast banner, org branding, user-manual screenshots

Third-party integrations (live)

Integration	What it does	Env vars
-------------	--------------	----------

QuickBooks Online	Pushes donations → Sales Receipts with donor + fund mapping. OAuth refresh, sync history.	QBO_CLIENT_ID, QBO_CLIENT_SECRET, QBO_REDIRECT_URI, QBO_ENVIRONMENT
IMAP/SMTP (Gmail, iCloud, Outlook, Yahoo, ProtonMail Bridge, custom)	Per-user inbox + send. Preset wizard with provider host/port pre-fill.	(per-user; stored encrypted in DB)
Google + Microsoft OAuth	Sign-in-with-Google / Sign-in-with-Microsoft for email accounts.	GOOGLE_OAUTH_*, MICROSOFT_OAUTH_*

Not integrated (documented in docs/INTEGRATIONS.md): Google Calendar, S3/Spaces, Twilio/SMS, on-platform payments.

Repo layout

```
furnish-hope/
├─ api/                                <- Node + Express + TypeScript
│   └─ src/
│       ├── routes/                   <- 38 files, 216 endpoints
│       └─ auth/                      <- migrations, audit, session,
middleware
│   ├── admin/                       <- metadata-driven generic admin
│   │   (introspect.ts)
│   ├── email/                       <- IMAP sync, OAuth, crypto
│   ├── quickbooks/                  <- OAuth + sync engine
│   └─ storage/                      <- pluggable attachment storage
(pg_blob today)
│   └─ db/pool.ts                    <- pg pool + withTransaction
├─ web/                              <- React + Vite + TypeScript +
Tailwind
│   └─ src/
│       ├── pages/                   <- 80 pages across staff / agency /
dev / admin / help
│       └─ components/               <- Sidebar, error boundary, issue
reporter,
│   │   broadcast banner, FK select with
create, etc.
│   └─ lib/                          <- API client, auth context, admin
metadata
│   └─ hooks/
├─ db/                               <- initial schema + seed + wipe + ERD
generator
├─ docs/                             <- 16 markdown docs (setup,
integrations, ERD regen)
│   + ERD PDF + user manual sources
├─ scripts/                          <- Python generators (ERD, codebase
summary, import templates)
├─ .do/app.yaml                      <- DigitalOcean deployment spec
└─ DEPLOY.md
```

Generic admin layer

The notable architectural piece. Every table can be edited at `/admin/<table>` through one metadata-driven form:

- **api/src/admin/introspect.ts** reads `information_schema` at startup to build a `SchemaMap` of every table, column, PK, FK, default, and nullability.
- **api/src/admin/config.ts** layers manual UX overrides (friendly labels, sidebar groups, enum value hints for CHECK-constrained VARCHARs, hide-in-form / hide-in-list flags).
- **web/src/pages/admin/AdminForm.tsx** renders any table's form by walking the metadata — FK fields become searchable dropdowns, enum VARCHARs become `<select>`s, columns with SQL defaults get omitted from INSERT when the user leaves them blank, etc.

That's how all ~160 tables get an editor without ~160 hand-written forms.

Operational features built on top of the admin layer

- **Per-entity widgets:** Email widget (your mail with this donor/client), Attachments widget (per-entity files), and Notes widget all attach to any detail page via `(entity_type, entity_id)`.
 - **Quick-create modals:** from any FK dropdown, "+ New ..." inline-creates the referenced row (Donor, Address, Vehicle, Contact, Facility, Staff, Pledge, Campaign).
 - **In-app User Manual** at `/help` with sticky TOC, anchor-aware Help buttons on every page header, retry-loop scroll for late-loading content, single-tab reuse.
 - **Issue reporter:** admin-only "Report issue" button captures a screenshot via `html2canvas`, posts to `/api/issues`. Developer console at `/dev/issues` triages with status workflow + broadcast banners.
 - **ERD PDF:** 12 themed pages, regenerated via `python scripts/generate_erd_pdf.py`, served from `/api/admin/erd`.
-

Tooling & dev workflow

- **Local dev:** two terminals — `api/ npm run dev` (tsx watch on :4000) + `web/ npm run dev` (Vite on :5173)
 - **TypeScript** — `npm run tsc --noEmit` for both; build verified via `vite build`
 - **No test suite** — verification is via `tsc + vite build + manual smoke tests + the in-app issue tracker`
 - **Deploy:** `git push origin main` → DigitalOcean rebuilds in 3-5 min → live at `hammerhead-app-tk838.ondigitalocean.app` (current dev URL)
 - **Docs:** 16+ markdown files under `docs/` cover setup (DO + Windows + Mac), integrations (QBO + email), ERD regen, user manual, OAuth setup, agency onboarding, third-party TODO list
-

What's notable about the design

- **No ORM** — direct parameterized SQL throughout. Schema is the

source of truth.

- **No tests** — coverage is from runtime checks (CHECK constraints, FK constraints, audit log) plus tsc + smoke tests + an in-app issue reporter for what slips through.
 - **Idempotent migrations** — every migration is wrapped to skip if the change is already present. Every startup re-runs them; the system self-heals.
 - **Email is per-user, not shared** — every staff member connects their own inbox. No shared mailbox.
 - **One generic admin form** handles every table; operational forms (Clients, Donors, Visits, etc.) provide a friendly UI for the most-used tables.
 - **Three shells** under one Express service share auth + session storage; deploy is a single artifact.
-

How this PDF is maintained

This file's source is docs/CODEBASE_SUMMARY.md and the rendered PDF lives at docs/CodebaseSummary.pdf. To regenerate after the codebase changes:

```
cd C:\Users\prest\furnish-hope
python scripts\generate_codebase_summary_pdf.py
git add docs\CODEBASE_SUMMARY.md docs\CodebaseSummary.pdf
git commit -m "docs: refresh codebase summary"
git push
```

The Python script reads the markdown source, runs the stat-collection queries above, refreshes the "By the numbers" table inline, then renders to PDF via Pandoc.